

Diseño Protocolo Sistema Archivos

Estudiante: Carlos Alvarado Meneses

Carné: C4C337

Universidad de Costa Rica

Facultad de Ingeniería

Escuela de Ciencias de la Computación e Informática

CI0123: PI Redes-SO

Prof. Francisco, Ariel, Maeva

Marzo, 2026

Índice

Introducción	1
Especificaciones del Lenguaje y Almacenamiento	1
Estructura del Archivo .dat	1
Dimensionamiento Fijo	2
Bitmap	2
Estructura del Bitmap	2
Funciones del Bitmap	2
Inodos	3
Estructura del Inodo	3
Funciones de la Tabla de Inodos	3
Bloques de Datos y Encadenamiento	3
Sistema de Encadenamiento	4
Representación de Figuras	4
Ejemplo de figura	4
Operaciones del Sistema	5
Creación de archivos	5
Lectura de archivos	5
Escritura de archivos	5
Eliminación de archivos	6
Interpretación del archivo .dat en C++	6
Cómo el código entiende el .dat	6

Resumen del Flujo de Ejecución

Índice de tablas

1.	Especificaciones generales del sistema	1
2.	Distribución del archivo .dat	1
3.	Funciones del Bitmap	2
4.	Estructura del inodo	3
5.	Funciones de la tabla de inodos	3
6.	Estructura interna del bloque de datos	4
7.	Resumen de flujos de operación	6

Introducción

El sistema de archivos está diseñado para ser implementado en el lenguaje C++ y utiliza un archivo con extensión `.dat` como simulador del dispositivo de almacenamiento.

El objetivo principal es contar con un sistema de archivos funcional que permita almacenar figuras representadas por caracteres asterisco (*), organizadas en bloques de tamaño fijo, con capacidad de encadenamiento para figuras que excedan la capacidad de un solo bloque.

Especificaciones del Lenguaje y Almacenamiento

Elemento	Especificación
Lenguaje de implementación	C++
Extensión del archivo	.dat
Tamaño máximo del archivo	256 KB
Tamaño del bloque	256 bytes
Representación de figuras	Carácter '*'

Cuadro 1

Especificaciones generales del sistema

Estructura del Archivo .dat

El archivo `.dat` simula el disco completo y se organiza de la siguiente manera:

Sección	Tamaño	Descripción
Bitmap	Fijo (según N bloques)	Indica bloque libre (0) u ocupado (1)
Tabla de inodos	Fijo ($N \text{ inodos} \times \text{tamaño inodo}$)	Metadatos de cada archivo
Bloques de datos	Variable (resto del espacio)	Datos + punteros (256 bytes c/u)

Cuadro 2

Distribución del archivo .dat

Dimensionamiento Fijo

El sistema utiliza tamaños fijos para el Bitmap y la tabla de inodos, lo que simplifica su implementación y permite un acceso rápido a estas estructuras. Para un disco de 256 KB con bloques de 256 bytes, se tienen 1024 bloques en total.

- **Bitmap:** 1024 bits = 128 bytes (ocupa parte del primer bloque)
- **Inodos:** Por ejemplo, 16 inodos $\times$ 16 bytes = 256 bytes (1 bloque)
- **Bloques de datos:** Restantes (aproximadamente 1023 bloques)

Bitmap

El Bitmap es una estructura que ocupa un espacio fijo al inicio del archivo `.dat`. Su función es registrar el estado de ocupación de cada bloque de datos.

Estructura del Bitmap

- Cada bit representa un bloque de datos.
- **Valor 0:** Bloque libre.
- **Valor 1:** Bloque ocupado.

Funciones del Bitmap

Función	Descripción
<code>marcarOcupado(bloque)</code>	Cambia el bit correspondiente a 1
<code>marcarLibre(bloque)</code>	Cambia el bit correspondiente a 0
<code>estaLibre(bloque)</code>	Verifica si un bloque específico está libre

Cuadro 3

Funciones del Bitmap

Inodos

Los inodos son estructuras de tamaño fijo que almacenan los metadatos de cada archivo. Se organizan en una tabla contigua dentro del archivo `.dat`.

Estructura del Inodo

Cada inodo tiene la siguiente composición:

Campo	Tamaño	Descripción
nombre	8 bytes	Nombre del archivo
tamaño	4 bytes	Tamaño total del archivo en bytes
primer_bloque	2 bytes	Número del primer bloque de datos
activo	1 byte	1 = archivo activo, 0 = inodo libre
reservado	1 byte	Para alineación a 16 bytes

Cuadro 4

Estructura del inodo

Funciones de la Tabla de Inodos

Función	Descripción
<code>crearInodo(nombre)</code>	Busca un inodo libre y lo inicializa
<code>buscarInodo(nombre)</code>	Localiza un inodo por nombre de archivo
<code>eliminarInodo(nombre)</code>	Marca el inodo como libre
<code>actualizarTamano(nombre, nuevoTam)</code>	Actualiza el tamaño del archivo

Cuadro 5

Funciones de la tabla de inodos

Bloques de Datos y Encadenamiento

Cada bloque de datos tiene un tamaño fijo de **256 bytes**. Su estructura interna se divide en dos secciones:

Sección	Tamaño	Descripción
Datos	240 bytes	Contenido de la figura (caracteres '*')
Puntero	16 bytes	Número del siguiente bloque o EOF

Cuadro 6

Estructura interna del bloque de datos

Sistema de Encadenamiento

El sistema de archivos implementa un mecanismo de **lista enlazada** para manejar figuras que exceden la capacidad de un solo bloque.

Bloque 5: [****...****] [-> 9]

Bloque 9: [****...****] [-> 12]

Bloque 12: [****...****] [EOF]

Figura 1

Ejemplo de encadenamiento de bloques

Cuando una figura pesa más de 240 bytes (datos útiles por bloque), el puntero apunta al siguiente bloque donde continúa el resto de la figura. El último bloque contiene un valor especial **EOF** (End Of File) en su campo puntero.

Representación de Figuras

Las figuras almacenadas en el sistema están compuestas exclusivamente por el carácter **asterisco (*)**. Estas figuras simulan siluetas de animales o formas geométricas.

Ejemplo de figura

```

*
***
*****
*****

```


Operaciones del Sistema

El programa en C++ será capaz de interpretar la estructura del archivo `.dat` y realizar las siguientes operaciones:

Creación de archivos

1. Buscar un inodo libre en la tabla de inodos.
2. Buscar un bloque libre en el Bitmap.
3. Marcar el bloque como ocupado en el Bitmap.
4. Inicializar el inodo con nombre, primer bloque y tamaño 0.
5. Inicializar el bloque con datos vacíos y puntero a EOF.

Lectura de archivos

1. Buscar el inodo por nombre de archivo.
2. Obtener el primer bloque desde el inodo.
3. Leer el contenido del bloque.
4. Seguir el puntero al siguiente bloque hasta encontrar EOF.
5. Mostrar la figura completa.

Escritura de archivos

1. Localizar el inodo del archivo.
2. Recorrer los bloques existentes.
3. Escribir en el área de datos disponible.
4. Si el bloque se llena, buscar un nuevo bloque libre.
5. Enlazar el nuevo bloque mediante el puntero.
6. Actualizar el tamaño en el inodo.

Eliminación de archivos

1. Buscar el inodo del archivo.
2. Recorrer todos los bloques enlazados.
3. Marcar cada bloque como libre en el Bitmap.
4. Marcar el inodo como libre.

Interpretación del archivo .dat en C++

El programa en C++ debe ser capaz de abrir el archivo `.dat` en modo binario y leer/escribir directamente en las posiciones correspondientes.

Cómo el código entiende el .dat

El programa interpreta el archivo `.dat` mediante:

1. **Desplazamientos (offsets) fijos:** Cada sección (Bitmap, inodos, bloques) tiene una posición conocida.
2. **Lectura por bloques:** Se leen exactamente 256 bytes a la vez.
3. **Estructuras en memoria:** Se utilizan `struct` para representar inodos y cabeceras de bloque.

Resumen del Flujo de Ejecución

Operación	Flujo
Lectura	Inodo → primer bloque → seguir punteros → EOF
Escritura	Inodo → llenar bloque → asignar nuevos bloques → actualizar
Eliminación	Inodo → recorrer bloques → liberar en Bitmap

Cuadro 7

Resumen de flujos de operación

Anexos

Anexo A: Convenciones de valores especiales

- **EOF:** Valor 0xFFFF (65535) en el campo puntero indica fin del archivo.
- **Inodo libre:** Campo activo = 0.
- **Bloque libre:** Bit correspondiente en Bitmap = 0.

Anexo B: Tamaños definitivos (ejemplo numérico)

- Tamaño del disco: 256 KB = 262,144 bytes.
- Tamaño del bloque: 256 bytes.
- Total de bloques: $262,144 / 256 = 1024$ bloques.
- Bitmap: 1024 bits = 128 bytes.
- Inodos: $16 \text{ inodos} \times 16 \text{ bytes} = 256 \text{ bytes}$.
- Bloques de datos: $1024 - 2 = 1022$ bloques (aprox).